

Exploratory Data Analysis

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

1. Load the data

```
In [3]: df=pd.read_csv('wdbc.data')
df.head()
```

```
Out[3]:
```

	842302	M	17.99	10.38	122.8	1001	0.1184	0.2776	0.3001	0.1471	...	25.38
0	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99
1	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57
2	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	14.91
3	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	22.54
4	843786	M	12.45	15.70	82.57	477.1	0.12780	0.17000	0.1578	0.08089	...	15.47

5 rows × 32 columns

```
In [4]: df.shape
```

```
Out[4]: (568, 32)
```

```
In [5]: columns = [
    "id", "diagnosis",
    "radius_mean", "texture_mean", "perimeter_mean", "area_mean",
    "smoothness_mean", "compactness_mean", "concavity_mean",
    "concave_points_mean", "symmetry_mean", "fractal_dimension_mean",

    "radius_se", "texture_se", "perimeter_se", "area_se",
    "smoothness_se", "compactness_se", "concavity_se",
    "concave_points_se", "symmetry_se", "fractal_dimension_se",

    "radius_worst", "texture_worst", "perimeter_worst", "area_worst",
    "smoothness_worst", "compactness_worst", "concavity_worst",
    "concave_points_worst", "symmetry_worst", "fractal_dimension_worst"
]
```

```
df = pd.read_csv("wdbc.data", header=None, names=columns)

df.head()
```

```
Out[5]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothn
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 32 columns

2. EDA

```
In [6]: df.shape
```

```
Out[6]: (569, 32)
```

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    569 non-null    int64
1   diagnosis                            569 non-null    object
2   radius_mean                          569 non-null    float64
3   texture_mean                         569 non-null    float64
4   perimeter_mean                       569 non-null    float64
5   area_mean                           569 non-null    float64
6   smoothness_mean                      569 non-null    float64
7   compactness_mean                     569 non-null    float64
8   concavity_mean                       569 non-null    float64
9   concave_points_mean                 569 non-null    float64
10  symmetry_mean                        569 non-null    float64
11  fractal_dimension_mean               569 non-null    float64
12  radius_se                            569 non-null    float64
13  texture_se                           569 non-null    float64
14  perimeter_se                         569 non-null    float64
15  area_se                              569 non-null    float64
16  smoothness_se                        569 non-null    float64
17  compactness_se                       569 non-null    float64
18  concavity_se                         569 non-null    float64
19  concave_points_se                   569 non-null    float64
20  symmetry_se                          569 non-null    float64
21  fractal_dimension_se                 569 non-null    float64
22  radius_worst                         569 non-null    float64
23  texture_worst                        569 non-null    float64
24  perimeter_worst                      569 non-null    float64
25  area_worst                           569 non-null    float64
26  smoothness_worst                     569 non-null    float64
27  compactness_worst                     569 non-null    float64
28  concavity_worst                      569 non-null    float64
29  concave_points_worst                 569 non-null    float64
30  symmetry_worst                       569 non-null    float64
31  fractal_dimension_worst              569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

```
In [8]: df.describe()
```

Out[8]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothnes
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	C
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	C
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	C
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	C
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	C
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	C
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	C

8 rows × 31 columns

In [9]: `df.isnull().sum()`

```
Out[9]: id                                0
        diagnosis                         0
        radius_mean                       0
        texture_mean                      0
        perimeter_mean                    0
        area_mean                         0
        smoothness_mean                   0
        compactness_mean                   0
        concavity_mean                     0
        concave_points_mean                0
        symmetry_mean                      0
        fractal_dimension_mean              0
        radius_se                          0
        texture_se                         0
        perimeter_se                       0
        area_se                           0
        smoothness_se                      0
        compactness_se                     0
        concavity_se                       0
        concave_points_se                  0
        symmetry_se                        0
        fractal_dimension_se                0
        radius_worst                       0
        texture_worst                      0
        perimeter_worst                    0
        area_worst                         0
        smoothness_worst                   0
        compactness_worst                   0
        concavity_worst                     0
        concave_points_worst                0
        symmetry_worst                      0
        fractal_dimension_worst             0
        dtype: int64
```

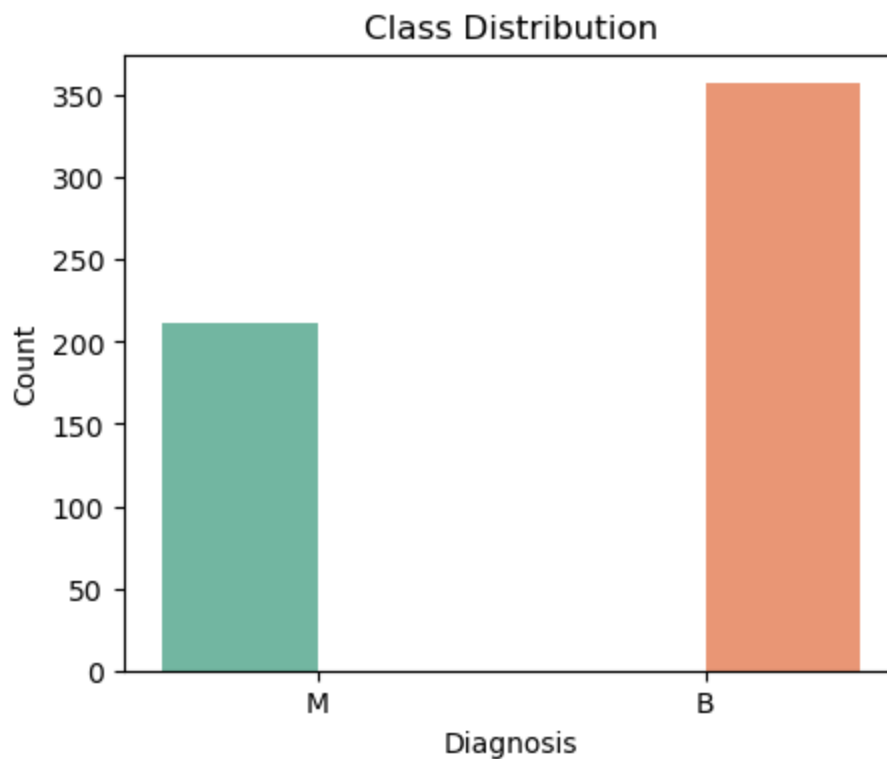
```
In [10]: df.duplicated().sum()
```

```
Out[10]: 0
```

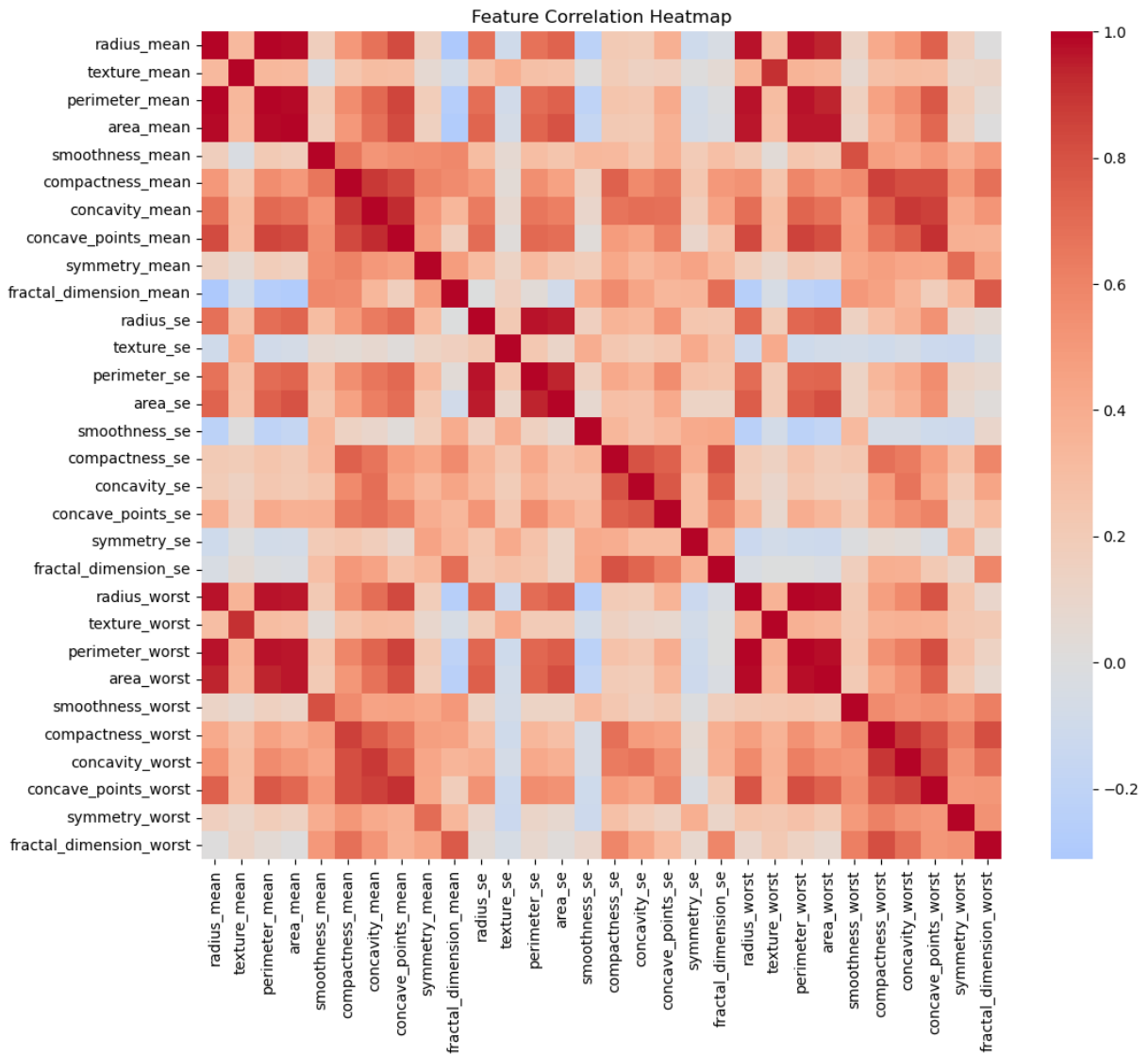
```
In [11]: df['diagnosis'].value_counts()
```

```
Out[11]: diagnosis
B      357
M      212
Name: count, dtype: int64
```

```
In [12]: plt.figure(figsize=(5, 4))
sns.countplot(data=df, x='diagnosis', hue='diagnosis', palette='Set2')
plt.title('Class Distribution')
plt.xlabel('Diagnosis')
plt.ylabel('Count')
plt.legend([], [], frameon=False)
plt.show()
```

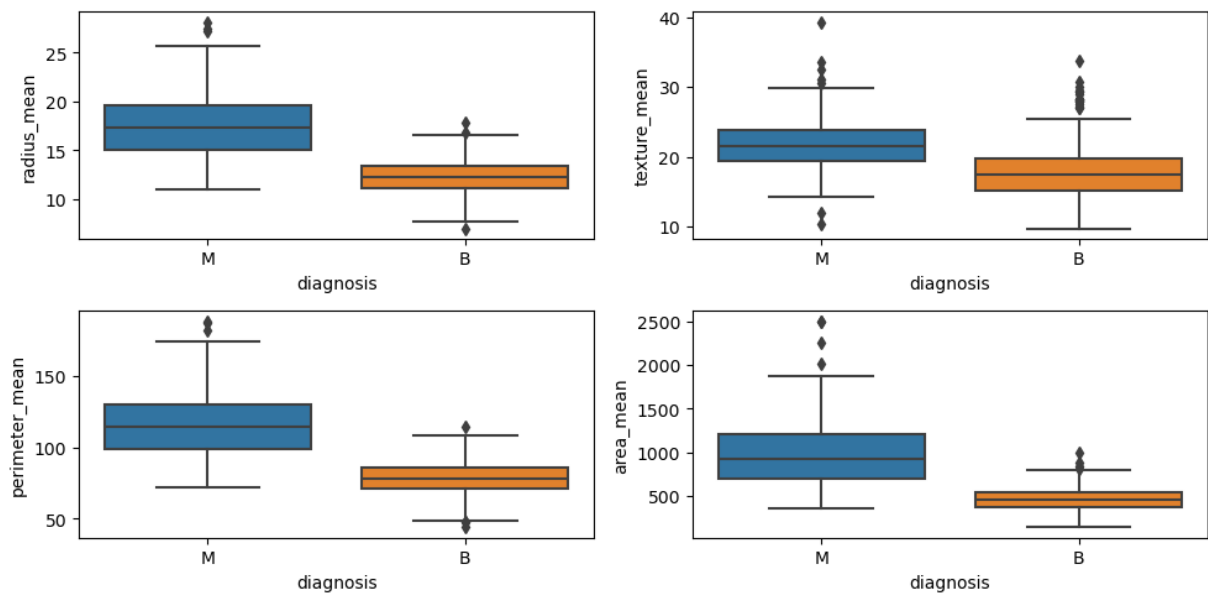


```
In [13]: plt.figure(figsize=(12, 10))
sns.heatmap(df.drop(columns=['id', 'diagnosis']).corr(), cmap='coolwarm', center=0)
plt.title('Feature Correlation Heatmap')
plt.show()
```

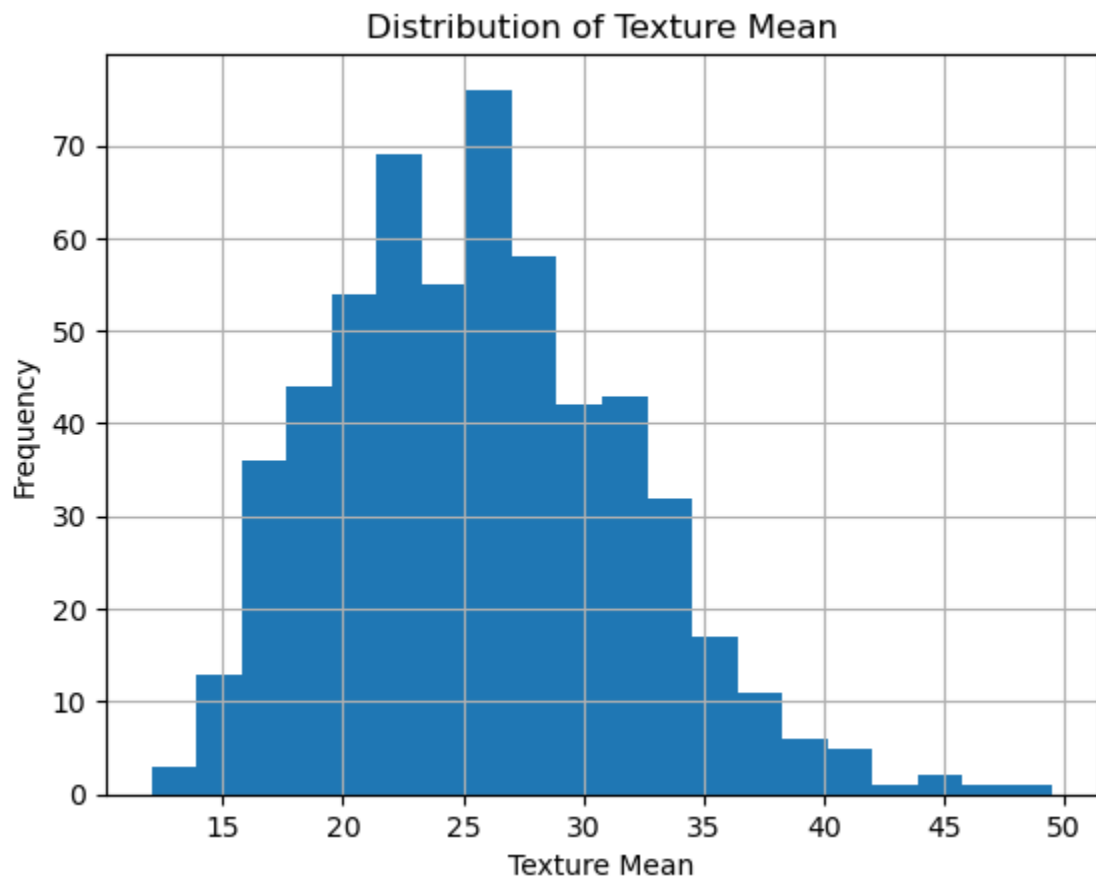


```
In [33]: features = ["radius_mean", "texture_mean", "perimeter_mean", "area_mean"]

plt.figure(figsize=(10,5))
for i, feature in enumerate(features, 1):
    plt.subplot(2,2,i)
    sns.boxplot(data=df, x="diagnosis", y=feature)
plt.tight_layout()
plt.show()
```



```
In [36]: df["texture_worst"].hist(bins=20)
plt.title("Distribution of Texture Mean")
plt.xlabel("Texture Mean")
plt.ylabel("Frequency")
plt.show()
```



3. Data Preprocessing

```
In [14]: X = df.drop(columns=['id', 'diagnosis'])
y = df['diagnosis'].map({'M': 1, 'B': 0})
```

```
In [15]: y.value_counts(normalize=True)
```

```
Out[15]: diagnosis
0    0.627417
1    0.372583
Name: proportion, dtype: float64
```

4. Test-Train split

```
In [16]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

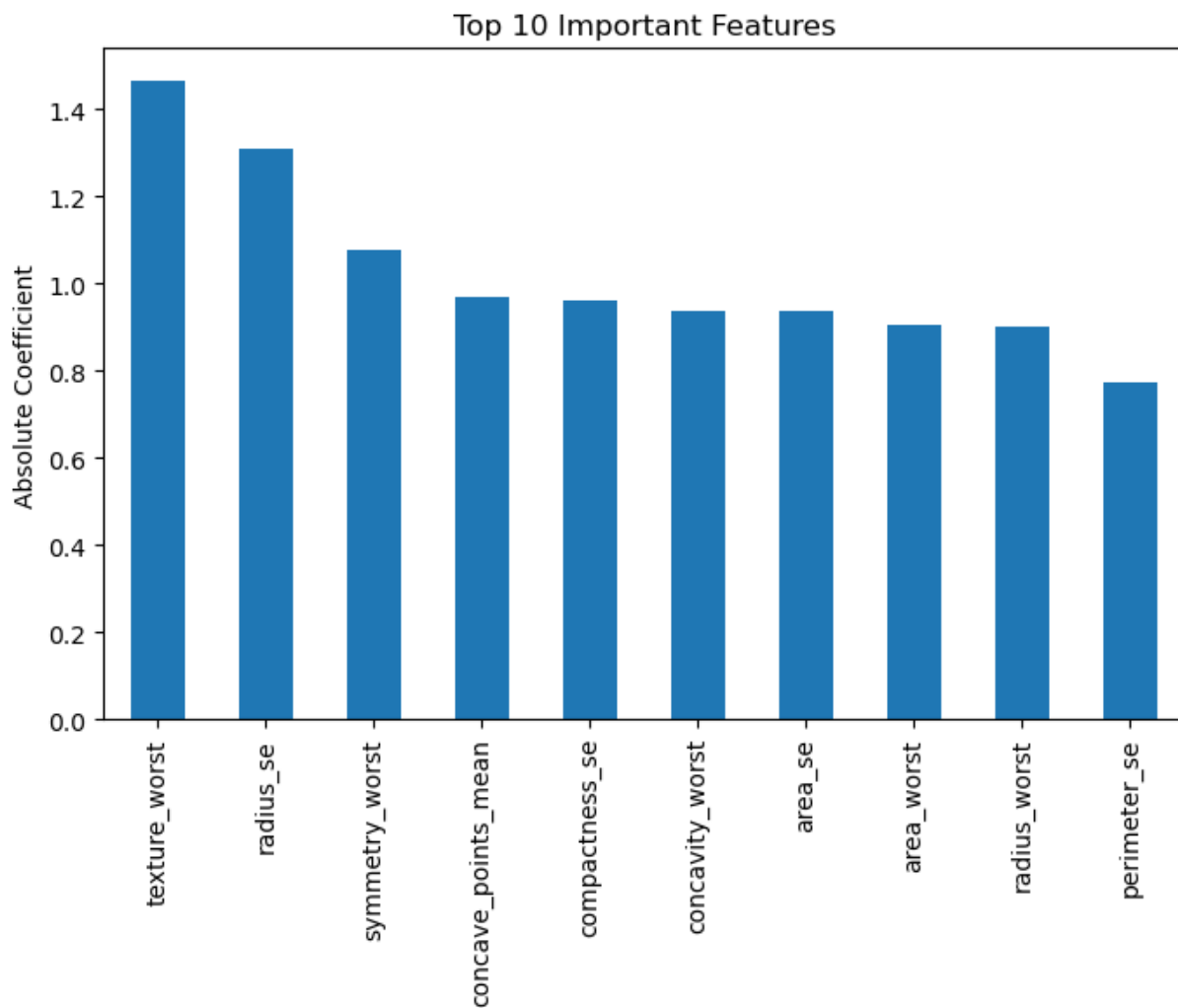
5. Logistic Regression

```
In [17]: log_reg = LogisticRegression(class_weight='balanced', random_state=42)
log_reg.fit(X_train_scaled, y_train)

y_pred_log_reg = log_reg.predict(X_test_scaled)
```

```
In [31]: coef = pd.Series(log_reg.coef_[0], index=X.columns)
coef = coef.abs().sort_values(ascending=False).head(10)

plt.figure(figsize=(8,5))
coef.plot(kind='bar')
plt.title("Top 10 Important Features")
plt.ylabel("Absolute Coefficient")
plt.show()
```



6. KNN

```
In [47]: from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

k_heuristic = int(np.sqrt(len(X_train)))
print("Heuristic K:", k_heuristic)

k_values = range(1, 16)
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train_scaled, y_train, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())

best_k = list(k_values)[np.argmax(cv_scores)]
print("Best K:", best_k)
print("Best CV Accuracy:", max(cv_scores))

knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train_scaled, y_train)
```

```
y_pred_knn = knn.predict(X_test_scaled)
```

Heuristic K: 21

Best K: 3

Best CV Accuracy: 0.9692307692307693

7. Model Evaluation

```
In [39]: def evaluate_model(y_true, y_pred):
          return {
              'Accuracy': accuracy_score(y_true, y_pred),
              'Precision': precision_score(y_true, y_pred, pos_label=1),
              'Recall': recall_score(y_true, y_pred, pos_label=1),
              'F1 Score': f1_score(y_true, y_pred, pos_label=1),
              'Confusion Matrix': confusion_matrix(y_true, y_pred)
          }

log_reg_results = evaluate_model(y_test, y_pred_log_reg)
knn_results = evaluate_model(y_test, y_pred_knn)
```

```
In [40]: print('Logistic Regression Confusion Matrix:')
          print(log_reg_results['Confusion Matrix'])
          print('\nLogistic Regression Classification Report:')
          print(classification_report(y_test, y_pred_log_reg, target_names=['Benign', 'Malignant']))
```

Logistic Regression Confusion Matrix:

```
[[71  1]
 [ 2 40]]
```

Logistic Regression Classification Report:

	precision	recall	f1-score	support
Benign	0.97	0.99	0.98	72
Malignant	0.98	0.95	0.96	42
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

```
In [41]: print('KNN Confusion Matrix:')
          print(knn_results['Confusion Matrix'])
          print('\nKNN Classification Report:')
          print(classification_report(y_test, y_pred_knn, target_names=['Benign', 'Malignant']))
```

KNN Confusion Matrix:

```
[[71  1]
 [ 6 36]]
```

KNN Classification Report:

	precision	recall	f1-score	support
Benign	0.92	0.99	0.95	72
Malignant	0.97	0.86	0.91	42
accuracy			0.94	114
macro avg	0.95	0.92	0.93	114
weighted avg	0.94	0.94	0.94	114

8. Comparison

```
In [42]: comparison_df = pd.DataFrame([
    {
        'Model': 'Logistic Regression',
        'Accuracy': log_reg_results['Accuracy'],
        'Precision': log_reg_results['Precision'],
        'Recall': log_reg_results['Recall'],
        'F1 Score': log_reg_results['F1 Score']
    },
    {
        'Model': 'KNN',
        'Accuracy': knn_results['Accuracy'],
        'Precision': knn_results['Precision'],
        'Recall': knn_results['Recall'],
        'F1 Score': knn_results['F1 Score']
    }
])

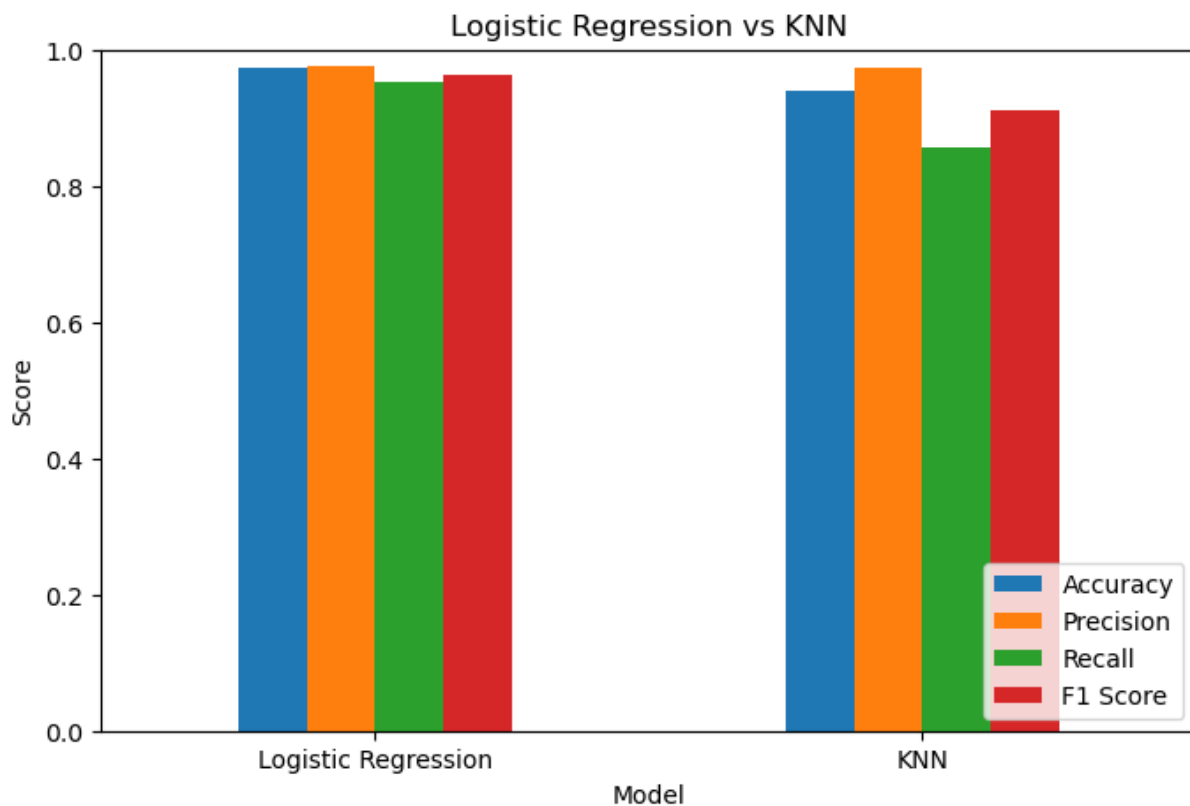
comparison_df.round(4)
```

```
Out[42]:
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.9737	0.9756	0.9524	0.9639
1	KNN	0.9386	0.9730	0.8571	0.9114

Visualization

```
In [43]: comparison_df.set_index('Model').plot(kind='bar', figsize=(8, 5))
plt.ylabel('Score')
plt.ylim(0, 1)
plt.title('Logistic Regression vs KNN')
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.show()
```

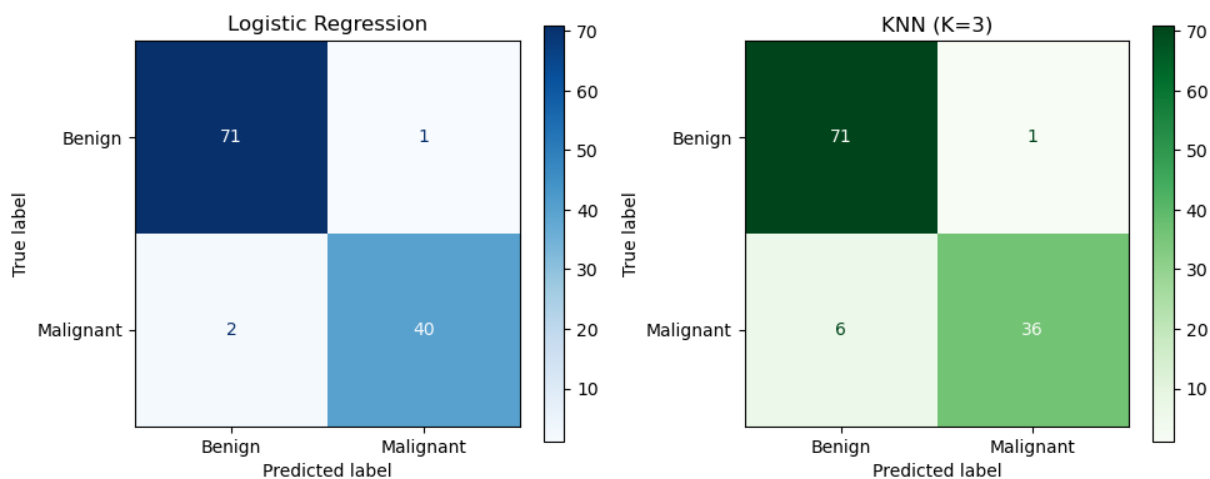


```
In [44]: fig, axes = plt.subplots(1, 2, figsize=(10, 4))

ConfusionMatrixDisplay.from_predictions(y_test, y_pred_log_reg, display_labels=['Ben
axes[0].set_title('Logistic Regression')

ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn, display_labels=['Benign
axes[1].set_title(f'KNN (K={best_k})')

plt.tight_layout()
plt.show()
```

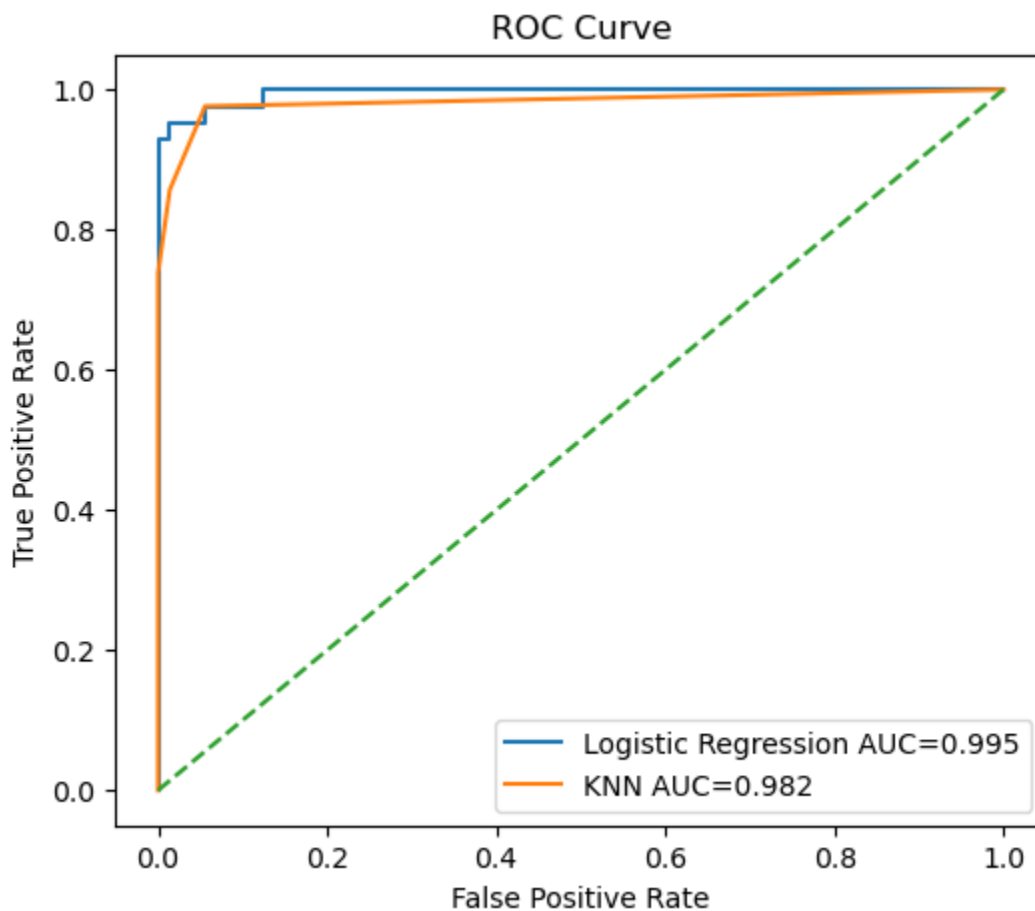


```
In [49]: from sklearn.metrics import roc_curve, roc_auc_score

log_prob = log_reg.predict_proba(X_test_scaled)[: , 1]
knn_prob = knn.predict_proba(X_test_scaled)[: , 1]

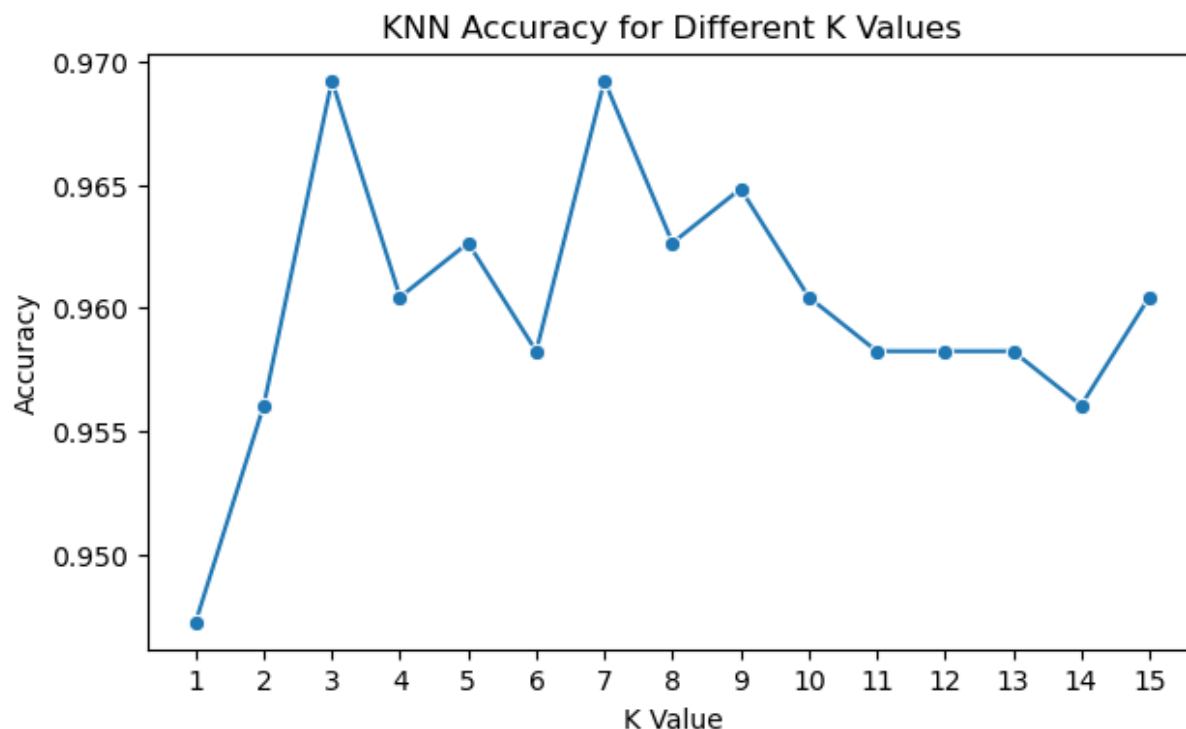
fpr_lr, tpr_lr, _ = roc_curve(y_test, log_prob)
fpr_knn, tpr_knn, _ = roc_curve(y_test, knn_prob)

plt.figure(figsize=(6, 5))
plt.plot(fpr_lr, tpr_lr, label=f'Logistic Regression AUC={roc_auc_score(y_test, log_prob):.3f}')
plt.plot(fpr_knn, tpr_knn, label=f'KNN AUC={roc_auc_score(y_test, knn_prob):.3f}')
plt.plot([0, 1], [0, 1], '--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```



```
In [48]: plt.figure(figsize=(7, 4))
sns.lineplot(x=list(k_values), y=cv_scores, marker='o')
plt.xticks(list(k_values))
plt.xlabel('K Value')
plt.ylabel('Accuracy')
plt.title('KNN Accuracy for Different K Values')
plt.show()
```

```
c:\Users\NILESH\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning:
use_inf_as_na option is deprecated and will be removed in a future version. Convert
inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
c:\Users\NILESH\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning:
use_inf_as_na option is deprecated and will be removed in a future version. Convert
inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



9. Final Interpretation

The overall performance of Logistic Regression was higher than the cross-validated KNN model in terms of accuracy, precision, recall and F1 score. The use of 5-fold cross-validation to select the optimal K instead of the test set improved the performance of KNN. Logistic Regression had a particularly high recall and this is important in the diagnosis of cancer as it minimises the risk of missing a malignant case. Hence, Logistic Regression is the preferred choice of classifier for this dataset.